

Practical



ID607001: Introductory Application Development Concepts

Assessment Information

Level	Credits	Assessment Type	Weighting
6	15	Individual	20%

Assessment Overview

In this individual assessment, you will write unit tests for an existing backend application - a **Quiz API** built with Express and Prisma. The application integrates with the [Open Trivia Database \(OpenTDB\)](#) API and supports two roles: **creator** (who creates and manages quizzes) and **player** (who plays quizzes).

You will not build the application from scratch. Instead, you will clone the provided starter repository, read and understand the existing code, and write unit tests that verify the correctness of its controllers and middleware using **Mocha**, **Chai** and **Sinon**.

Learning Outcome

At the successful completion of this course, learners will be able to:

1. Design and build secure applications with dynamic database functionality following an appropriate software development methodology.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Practical	20%	22 May at 4.59 PM	1

Assessment	Weighting	Due Date	Learning Outcome
Project	80%	26 June at 4.59 PM	1

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment must be completed by **22 May at 4.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of 50% across all assessments in ID607001: Introductory Application Development Concepts.

Submission

You must submit all application files via GitHub Classroom.

- Repository URL: You will be emailed a link to accept the GitHub Classroom assignment. After accepting, you will have your own copy of the repository where you will complete the assessment.
- Branch: Use the **main** branch for development and submission. You may create additional branches for testing purposes, but only the **main** branch will be marked.
- Git Ignore: If you do not have one, create a **.gitignore** using this resource - [Node.gitignore](#)
- Due Date: 22 May at 4.59 PM
- Late Penalty: 10% per day, rolling over at 12.00 AM

The latest application files in the **main** branch will be used to mark against the marking rubric. Please test your applications before you submit. Partial marks may be given for incomplete functionality.

Authenticity

All parts of your submitted assessment must be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

AI Tools

Learning to use AI tools is an important skill. While AI tools are powerful, you must be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You must still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

This also applies to code snippets retrieved from StackOverflow and GitHub.

Failure to do this may result in a mark of zero for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with Otago Polytechnic policies. Learners can view policies on the Otago Polytechnic website located at <https://www.op.ac.nz/about-us/governance-and-management/policies>.

Extensions

Familiarise yourself with the assessment due date. Extensions will only be granted if you are unable to complete the assessment by the due date because of unforeseen circumstances outside your control. The length of the extension granted will depend on the circumstances and must be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments are not applicable in ID607001: Introductory Application Development Concepts.

Application Overview

The starter repository contains a fully implemented **Quiz API**. Before writing any tests, spend time reading through the codebase to understand how it is structured and how the different parts work together.

Domain Description

The application supports two types of users:

- **Creator** - can create and manage quiz categories, quizzes, and questions. Creators can import questions directly from the OpenTDB API into a quiz.
- **Player** - can browse available quizzes, start a game session, submit answers, and view their score history.

Models

The application includes the following six models:

Model	Description
User	A registered user with a role of either PLAYER or CREATOR
Category	A topic area for quizzes (e.g. Science, History), created by a creator
Quiz	A quiz belonging to a category, created by a creator
Question	A multiple-choice question belonging to a quiz, with one correct answer and three incorrect answers
GameSession	A player's attempt at completing a quiz
Answer	A player's submitted answer to a question within a game session

Project Structure

```
backend/
├── .mocharc.json
├── .c8rc
├── controllers/
│   ├── auth.js
│   ├── category.js
│   ├── quiz.js
│   ├── question.js
│   ├── gameSession.js
│   └── answer.js
├── middleware/
│   ├── jwtAuth.js
│   └── rbac.js
├── repositories/
│   ├── category.js
│   ├── quiz.js
│   ├── question.js
│   ├── gameSession.js
│   └── answer.js
└── tests/
    ├── mocks/
    │   ├── category.mock.js
    │   ├── quiz.mock.js
    │   ├── question.mock.js
    │   ├── gameSession.mock.js
    │   └── answer.mock.js
    └── unit/
        ├── 00-auth.test.js
        ├── 01-category.test.js
        ├── 02-quiz.test.js
        ├── 03-question.test.js
        ├── 04-gameSession.test.js
        ├── 05-answer.test.js
        ├── 06-jwtAuth.test.js
        └── 07-rbac.test.js
```

Testing Tools

Library	Role
Mocha	Test runner - organises tests into suites and runs them
Chai	Assertion library - verifies that values match expectations
Sinon	Mocking library - replaces real dependencies with controlled fakes
c8	Code coverage - measures how much code your tests execute

Assessment Requirements - Unit Tests

The marking rubric is available [here](#).

Setup (not marked)

1. Clone the provided starter repository from GitHub Classroom and open it in Visual Studio Code.
2. Install dependencies: `npm install`.
3. Confirm the following scripts are present in `package.json`:

```
"test": "mocha",  
"test:coverage": "c8 mocha",  
"test:coverage:report": "c8 report --reporter=html"
```

4. Confirm `.mocharc.json` exists in `backend/` with the following content:

```
{  
  "spec": "tests/unit/**/*.test.js",  
  "timeout": 10000,  
  "exit": true  
}
```

5. Read through the `controllers/`, `middleware/` and `repositories/` directories before writing any tests. Understanding the code you are testing is essential.
-

Shared Mock Helpers (2 marks)

Before writing any test suites, create shared mock helpers in the `tests/mocks/` directory. Each file must own the helpers for one domain and be imported into the corresponding test file rather than redefining the helpers each time.

- Create a mock helper file for each of the five models that have a repository (`category`, `quiz`, `question`, `gameSession`, `answer`). Each file must export a `mockReq`, `mockRes` and a repository stub factory for that domain. (1 mark)
- Each `mockRes` must implement `res.status` as a Sinon stub that returns `res` itself, so that chained calls like `res.status(201).json(...)` work correctly. (1 mark)

Here is the expected shape of each mock file:

```
import sinon from "sinon";  
import someRepository from "../../repositories/some.js";  
  
export const mockReq = (body = {}, params = {}, query = {}) => ({  
  body,  
  params,  
  query,  
});
```

```
export const mockRes = () => {
  const res = {};
  res.status = sinon.stub().returns(res);
  res.json = sinon.stub().returns(res);
  return res;
};

export const stubSomeRepo = () => ({
  create: sinon.stub(someRepository, "create"),
  findAll: sinon.stub(someRepository, "findAll"),
  findById: sinon.stub(someRepository, "findById"),
  update: sinon.stub(someRepository, "update"),
  delete: sinon.stub(someRepository, "delete"),
});
```

Auth Controller Tests (4 marks)

Create `tests/unit/00-auth.test.js`. Use `afterEach(() => sinon.restore())` to reset all stubs between tests.

Your tests must cover:

- **Register - success:** the user does not already exist, returns `201`, and the response body does not include a `password` field. (1 mark)
- **Register - duplicate user:** the repository indicates the user already exists, returns `409`. (1 mark)
- **Login - success:** credentials are valid, returns `200` and the response body includes a `token` field. (1 mark)
- **Login - invalid credentials:** the user is not found or the password does not match, returns `401`. (1 mark)

Category Controller Tests (2 marks)

Create `tests/unit/01-category.test.js`. Use `afterEach(() => sinon.restore())`. Import your shared helpers from `tests/mocks/category.mock.js`.

Your tests must cover all five CRUD operations:

- **Create** - repository resolves with a new category, returns `201`. (0.5 marks)
 - **Read all** - repository resolves with a non-empty array, returns `200`; repository resolves with an empty array, returns `404`. (0.5 marks)
 - **Read by ID** - repository resolves with a category, returns `200`; repository resolves with `null`, returns `404`. (0.5 marks)
 - **Update** - existing category is found and updated, returns `200`; category is not found, returns `404`. (0.25 marks)
 - **Delete** - existing category is found and deleted, returns `200`; category is not found, returns `404`. (0.25 marks)
-

Quiz Controller Tests (2 marks)

Create `tests/unit/02-quiz.test.js`. Use `afterEach(() => sinon.restore())`. Import your shared helpers from `tests/mocks/quiz.mock.js`.

Your tests must cover all five CRUD operations following the same pattern as the category tests above. (2 marks)

Note that a quiz belongs to a category. In your tests you do not need a real category - stub the repository so it behaves as if one exists.

Question Controller Tests (2 marks)

Create `tests/unit/03-question.test.js`. Use `afterEach(() => sinon.restore())`. Import your shared helpers from `tests/mocks/question.mock.js`.

In addition to the standard five CRUD operations (1 mark), your tests must also cover the OpenTDB import endpoint:

- **Import from OpenTDB - success:** stub `fetch` so it resolves with a valid OpenTDB response (`response_code: 0`) containing at least one question object, returns `201` and includes a count of questions created in the response body. (0.5 marks)
 - **Import from OpenTDB - API failure:** stub `fetch` so it resolves with a non-zero `response_code`, returns an appropriate error status. (0.5 marks)
-

Game Session Controller Tests (2 marks)

Create `tests/unit/04-gameSession.test.js`. Use `afterEach(() => sinon.restore())`. Import your shared helpers from `tests/mocks/gameSession.mock.js`.

Your tests must cover:

- **Start session - success:** a valid player and quiz exist, repository creates a new session, returns `201`. (0.5 marks)
 - **Start session - quiz not found:** quiz repository resolves with `null`, returns `404`. (0.5 marks)
 - **Finalise session - success:** all answers are retrieved and the score percentage is calculated correctly, returns `200` with the score in the response body. (0.5 marks)
 - **Finalise session - session not found or already finalised:** repository resolves with `null` or a session that is already complete, returns an appropriate error status. (0.5 marks)
-

Answer Controller Tests (2 marks)

Create `tests/unit/05-answer.test.js`. Use `afterEach(() => sinon.restore())`. Import your shared helpers from `tests/mocks/answer.mock.js`.

Your tests must cover:

- **Submit answer - success:** the answer is recorded and the response body indicates whether the submitted answer was correct, returns `201`. (1 mark)

- **Submit answer - session not found:** game session repository resolves with `null`, returns `404`. (0.5 marks)
 - **Submit answer - question not found:** question repository resolves with `null`, returns `404`. (0.5 marks)
-

JWT Middleware Tests (2 marks)

Create `tests/unit/06-jwtAuth.test.js`. Import the middleware function directly and call it with mock `req`, `res` and `next` objects. Use `sinon.stub()` for `next` so you can assert whether it was called.

Your tests must cover:

- No `Authorization` header is present - `next` is not called, returns `401`. (0.5 marks)
- `Authorization` header is present but does not begin with `Bearer` - returns `401`. (0.5 marks)
- A valid, correctly signed token is provided - `next` is called with no arguments and `req.user` is set to the decoded payload. (0.5 marks)
- An expired or tampered token is provided - `next` is not called, returns `401`. (0.5 marks)

For the valid token test, sign a real token using `jwt.sign()` with the same secret your middleware uses rather than mocking the JWT library itself.

RBAC Middleware Tests (2 marks)

Create `tests/unit/07-rbac.test.js`. Import the middleware directly and call it with mock `req`, `res` and `next` objects. Use `sinon.stub()` for `next`.

Your tests must cover:

- `req.user` is undefined - `next` is not called, returns `403`. (0.5 marks)
 - The user's role is not in the permitted list - `next` is not called, returns `403`. (0.5 marks)
 - The user's role is in the permitted list - `next` is called with no arguments. (0.5 marks)
 - The middleware is configured with multiple permitted roles - a user with any matching role causes `next` to be called. (0.5 marks)
-

Code Coverage (2 marks)

Ensure `backend/.c8rc` exists with the following configuration:

```
{
  "reporter": ["text", "html"],
  "include": ["controllers/**/*.js", "middleware/**/*.js"],
  "exclude": ["tests/**", "prisma/**", "node_modules/**"],
  "branches": 80,
  "lines": 80,
  "functions": 80,
  "statements": 80,
  "all": true
}
```


Run `npm run test:coverage`. Your test suite must achieve:

- At least **80% branch coverage** across all included files. (1 mark)
- At least **80% statement and function coverage** across all included files. (1 mark)

Where a line is genuinely untestable (e.g. `app.listen()`), you may suppress it with `/* c8 ignore next */`. Each use must be accompanied by a comment explaining why that line cannot be tested.

Code Quality and Best Practices (not separately marked)

Your test code is subject to the same code quality expectations as the rest of the codebase:

- Use `afterEach(() => sinon.restore())` in every test suite to prevent stub state from leaking between tests.
 - Each `it()` description must read as a plain English sentence that clearly states what is being tested and what the expected outcome is.
 - Every test must follow the **Arrange / Act / Assert** pattern - set up stubs, call the function under test, then assert the result.
 - Do not write tests that pass regardless of the implementation (e.g. asserting `true` is `true`).
 - Code must be formatted using Prettier and linted using ESLint before submission.
-

Author: Grayson Orr

Course: ID607001: Introductory Application Development Concepts